

原始请求 · curl	Raw HTTP
基础 messages 请求	
<pre>curl https://api.anthropic.com/v1/messages \ -H "x-api-key: \$ANTHROPIC_API_KEY" \ -H "anthropic-version: 2023-06-01" \ -H "Content-Type: application/json" \ -d '{ "model": "claude-opus-4-7", "max_tokens": 1024, "system": "You are concise.", "messages": [{"role": "user", "content": "Hello"}] '</pre>	
流式 SSE	
<pre>curl .../v1/messages \ -H "x-api-key: \$KEY" \ -H "anthropic-version: 2023-06-01" \ -d '{ "model": "claude-sonnet-4-6", "max_tokens": 1024, "messages": [...], "stream": true '</pre>	
# 事件: message_start, content_block_start, content_block_delta, content_block_stop, message_delta, message_stop	
关键 header / 端点	
POST /v1/messages	主端点(非 chat/completions)
x-api-key	认证(非 Bearer)
anthropic-version: 2023-06-01	必传版本固定
anthropic-beta: ...	开 beta 特性
POST /v1/messages/count_tokens	计算 token
POST /v1/messages/batch	批处理
POST /v1/files	文件上传

Tool Use	工具调用
定义工具(注意 input_schema)	
<pre>tools = [{"name": "get_weather", "description": "Get city weather", "input_schema": { "type": "object", "properties": { "city": {"type": "string"} }, "required": ["city"] } }]</pre>	
控制 tool_choice	
{"type": "auto"}	默认
{"type": "any"}	必须用某工具
{"type": "tool", "name": "指定"}	指定
{"type": "none"}	禁用
disable_parallel_tool_use = true	禁止并行
回灌结果	
<pre>messages.append({"role": "assistant", "content": resp.content}) # 含 tool_use messages.append({"role": "user", "content": [{ "type": "tool_result", "tool_use_id": tu.id, "content": str(result), "is_error": false }] })</pre>	
stop_reason	
end_turn	正常完成
tool_use	需调工具
max_tokens	达上限
stop_sequence	命中停止

消息结构	Schema
独立 system 字段(非 role)	
<pre>{ "system": "string 或数组", "messages": [{"role": "user", "content": "文本或 blocks"}, {"role": "assistant", "content": "..."}] }</pre>	
content blocks(数组形态)	
<pre>{ "type": "text", "text": "文本..." }, { "type": "image", "source": "图片 {...}" }, { "type": "document", "source": "PDF 文档 {...}" }, { "type": "tool_use", "id": "工具调用(模型出)..." }, { "type": "tool_result", "tool_use_id": "工具结果(用户返回)..." }, { "type": "thinking", "text": "模型思考内容..." }</pre>	
必传参数	
model	如 claude-opus-4-7
max_tokens	必传(OpenAI 可省)
messages	轮次必须 user 起, user/assistant 交替
采样	
temperature / top_p	采样三件套
stop_sequences	停止序列数组

高级特性	Caching · Thinking · Vision
Prompt Caching(显式 · 90% 折扣)	
<pre>{ "system": [{"type": "text", "text": "长系统提示..."}, {"cache_control": {"type": "ephemeral"}}] }</pre>	
# 默认 5min; 1h 缓存: # "cache_control": {"type": "ephemeral", "ttl": "1h"} # 最多 4 个 breakpoint	
Extended Thinking	
<pre>resp = client.messages.create(model="claude-opus-4-7", max_tokens=16000, thinking={"type": "enabled", "budget_tokens": 10000}, messages=[...],) # resp.content 含 thinking + text blocks</pre>	
Vision · 图片	
<pre>{ "role": "user", "content": [{"type": "image", "source": { "type": "base64", "media_type": "image/png", "data": "..."}, }, {"type": "text", "text": "What is this?"}] }</pre>	
PDF / 文档	
<pre>{ "type": "document", "source": "PDF 文件", "citations.enabled": true }</pre>	
其他	
Files API	/v1/files 上传复用
Batch API	50% 折扣 · 24h

Python SDK · 同步	anthropic
安装 & 客户端	
<pre>pip install anthropic from anthropic import Anthropic client = Anthropic(api_key="sk-ant-...", timeout=60.0, max_retries=2,)</pre>	
基本调用	
<pre>resp = client.messages.create(model="claude-opus-4-7", max_tokens=1024, system="You are concise.", messages=[{"role": "user", "content": "Hi"}],) print(resp.content[0].text) print(resp.usage.input_tokens, resp.usage.output_tokens)</pre>	
流式(推荐 helper)	
<pre>with client.messages.stream(model="claude-sonnet-4-6", max_tokens=1024, messages=[...],) as stream: for text in stream.text_stream: print(text, end="", flush=True) final = stream.get_final_message()</pre>	
原始事件流	
client.messages.create	迭代 raw 事件对象
event.type	message_start / *_delta / *_stop

错误 & 网关	Errors & Gateways
错误码	
401 authentication_error	key 错
403 permission_error	无权限
404 not_found_error	model/资源
413 request_too_large	超 token 限
429 rate_limit_error	速率限
529 overloaded_error	服务过载, 重试
500 api_error	服务端
速率限 header	
<pre>anthropic-ratelimit-requests-remaining: 10 anthropic-tokens-remaining: 10000 anthropic-retry-after: 60</pre>	
官方网关	
AnthropicBedrock	AWS Bedrock 客户端
AnthropicVertex	GCP Vertex AI
<pre>from anthropic import AnthropicBedrock client = AnthropicBedrock(aws_region="us-west-2",)</pre>	
第三方/自建	
LiteLLM	统一 OpenAI/Anthropic 接口
claude-code-router	把 Claude Code 路由到任意模型
自建 base_url	改 Anthropic(base_url=...)

Python SDK · 异步	AsyncAnthropic
异步客户端	
<pre>import asyncio from anthropic import AsyncAnthropic client = AsyncAnthropic(api_key="sk-ant-...") async def ask(q): r = await client.messages.create(model="claude-haiku-4-5", max_tokens=512, messages=[{"role": "user", "content": q}],) return r.content[0].text asyncio.run(ask("hi"))</pre>	
并发 + 限流	
<pre>sem = asyncio.Semaphore(10) async def bound(q): async with sem: return await ask(q) results = await asyncio.gather(*[bound(q) for q in qs])</pre>	
异步流	
<pre>async with client.messages.stream(model="claude-sonnet-4-6", max_tokens=1024, messages=[...],) as stream: async for text in stream.text_stream: print(text, end="", flush=True)</pre>	
资源回收	
await client.close()	关 httpx 池
async with AsyncAnthropic(...)	上下文管理推荐